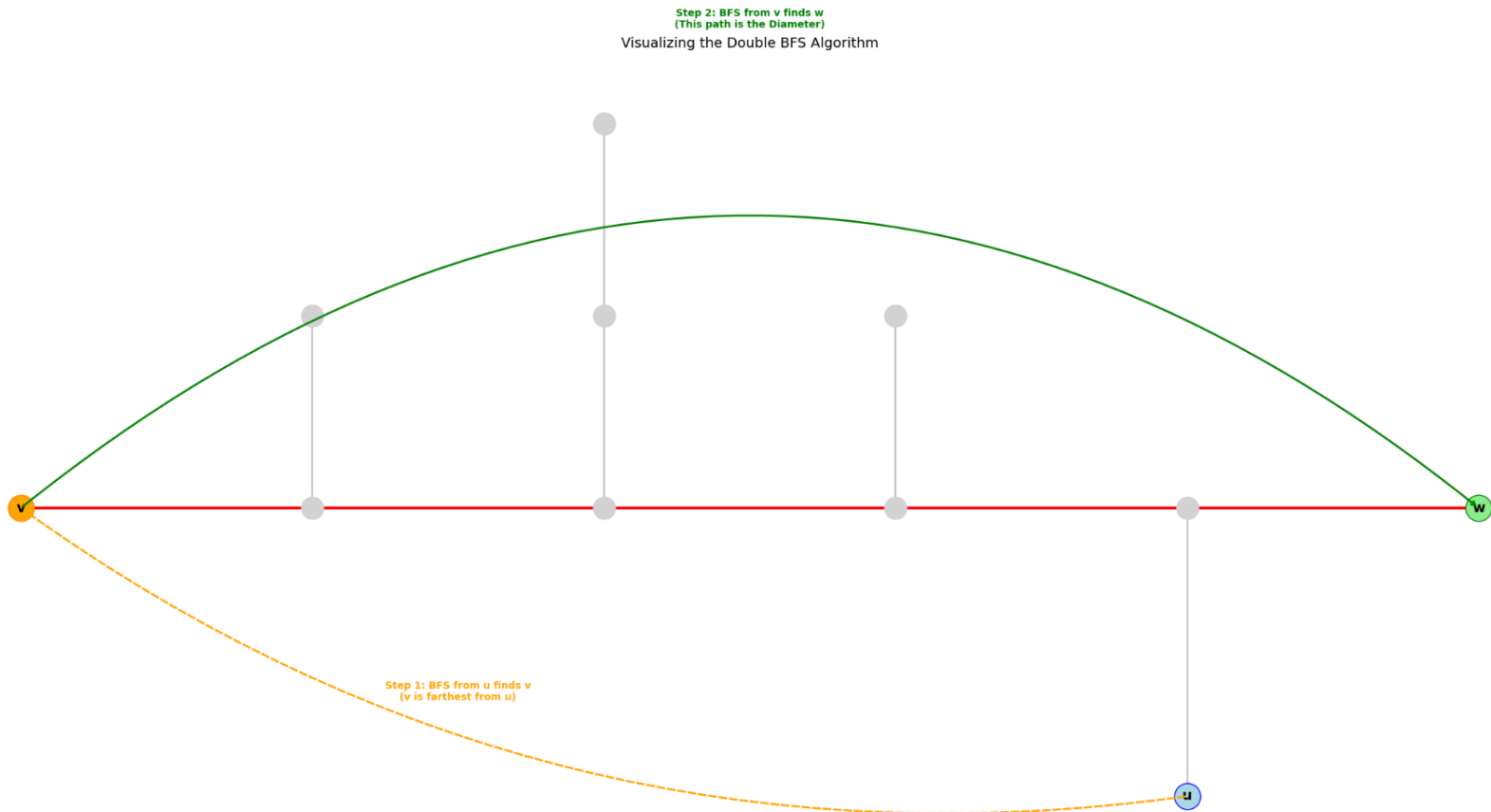


### Algorithm:

1. First is to find an arbitrary node  $u$
2. Then running BFS from  $u$  to find the farthest node  $v$
3. Running a second BFS starting from  $v$  to find the farthest node  $w$
4. You will get the tree's diameter, which is the distance between  $v$  and  $w$



### Implementation:

The algorithm was implemented using Python with the networkx library to generate the random labeled trees. BFS utilizes collections.deque for efficient queue operations. Refer to the attached tree\_diameter.txt for the full source code. Note: you will have to change it to a .py to run it, though, due to .py being blocked on Outlook.

### Proof of Correctness:

**Theorem:** The Double BFS algorithm correctly identifies the diameter of a tree. This relies on two properties:

1. For any arbitrary node  $u$ , the node  $v$  farthest from it is an endpoint of the tree's diameter
2. The standard BFS from a diameter endpoint determines the longest path in the tree

### Proof (Part 1): The First BFS finds an Endpoint

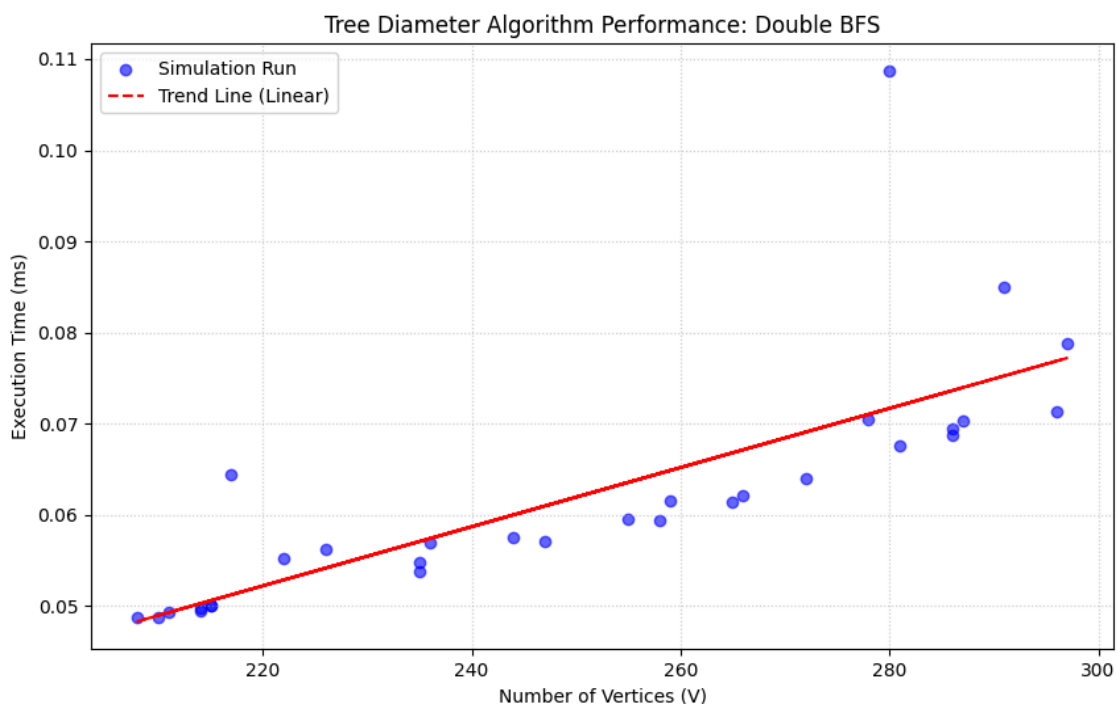
Let the actual diameter of the tree be the unique simple path between nodes  $x$  and  $y$ . We start our BFS at an arbitrary node  $u$  and find that  $v$  is the farthest node from  $u$ .

**Contradiction Assumption:** Assume  $v$  is not an endpoint of the diameter.

- **Case A (Disjoint Paths):** The path from  $u$  to  $v$  does not share any vertices with the diameter path  $x - y$ . In a tree, there is exactly one unique simple path between any two nodes. If any of these paths were really disjoint then the graph would be either disconnected, or else have to use a cycle to connect them, both violations of the definition of a tree. Thus, the paths must intersect.
- **Case B (Intersecting Paths):** The path from  $u$  to  $v$  crosses the diameter  $x - y$  at some node  $z$ . Since  $v$  is the farthest node from  $u$ , we have that  $d(u, v) \geq d(u, x)$ . We subtract the common segment of path  $u \rightarrow z$  and obtain  $d(z, v) \geq d(z, x)$ . Then, construct a path from diameter endpoint  $y$  to  $z$  and from  $z$  to  $v$ : its length is  $d(y, z) + d(z, v)$ . Since  $d(z, v) \geq d(z, x)$ , path  $y \rightarrow v$  is at least as long as diameter  $y \rightarrow x$ ,  $v$  is at least as far away as the true endpoint  $x$ , so  $v$  is a valid endpoint of the diameter.

### Proof (Part 2): The Second BFS finds the Diameter

From Part 1, we established that  $v$  is an endpoint of the diameter. We define the diameter as the longest separation between any two nodes in the tree. One end of this longest path is anchored at  $v$ , so the longest path from  $v$  must necessarily reach the other endpoint of the diameter: namely  $w$ . A standard BFS from  $v$  explores all reachable nodes and is guaranteed to find the node  $w$  which is at maximum distance, giving us the correct length of the diameter:  $d(v, w)$ .



This scatter plot shows the execution time (ms) of the double BFS algorithm on randomly generated trees ranging from 200 to 300 vertices. The red dashed trend line represents the expected linear time complexity  $O(V)$

**Complexity Analysis:**

**Time Complexity:**  $O(V + E)$ . Since it is a tree,  $E = V - 1$ . The complexity is  $O(V)$  (Linear). We run BFS twice, so it is strictly  $2 \times O(V)$

**Space Complexity:**  $O(V)$  to store the adjacency list and the visited set/queue during BFS